



A polynomial-time exact algorithm for the sectionalizing switch allocation problem

Fábio Luiz Usberti ^a,*, José Federico Vizcaino González ^b, Laura Silva de Assis ^c,
Celso Cavellucci ^a

^a Institute of Computing, Campinas State University, Av. Albert Einstein 1251, 13083-852, Campinas, SP, Brazil

^b Faculty of Engineering and Sciences, São Paulo State University, Av. Dr. Ariberto Pereira da Cunha 333, 12516-410, Guaratinguetá, SP, Brazil

^c Federal Center of Technological Education of Rio de Janeiro, Av. Maracanã 229, 25620-003, Rio de Janeiro, RJ, Brazil

ARTICLE INFO

Keywords:

Power distribution
Reliability
Dynamic programming
Combinatorial optimization

ABSTRACT

The allocation of switches in power distribution networks is a critical combinatorial optimization problem concerning reliability optimization. In this work, we consider a set of sectionalizing switches and a radial network, for which the objective is to find the best edges to allocate the switches to minimize the expected energy not supplied. An open question in the literature concerns the computational complexity of this fundamental problem, specifically, whether it is NP-hard. In this paper, we show that it is in fact tractable by presenting the first exact polynomial-time algorithm, based on dynamic programming. We compare our approach with previous state-of-the-art methodologies. Extensive computational experiments show that the proposed dynamic programming scales much better than the previous approach. Large instances, with more than three thousand nodes, are solved for the first time for any number of switches.

1. Introduction

The Switch Allocation Problem (SAP) is an overarching model that encompasses several reliability optimization problems explored in the literature. The general idea is to determine optimal switch locations within a distribution system to minimize the impact of service interruptions, typically measured by indices such as interruption frequency, duration, unsupplied power, or associated costs.

The seminal work by Levitin et al. [1] introduced a genetic algorithm for allocating sectionalizers in a radial distribution network, marking the first study on the SAP. Since then, various SAP formulations have emerged, including tie switch allocation [2], remote-controlled switch placement [3], protection device allocation [4], and fault indicator positioning [5]. The objectives guiding switch allocation differ as well, targeting reliability indices such as SAIDI, ENS, SAIFI, and Customer Interruption Costs [6,7], with some studies addressing multiple objectives simultaneously [8,9].

The methodologies used in SAP can be broadly divided into two main approaches: heuristics and exact algorithms. Heuristics are methods that, while not guaranteeing optimality, can produce sufficiently good solutions within polynomial time. Common heuristics applied to SAP include ant colony optimization [10], simulated annealing [11], bee colony optimization [2], genetic algorithms [12], memetic algorithms [13], and particle swarm optimization [14].

On the other hand, exact approaches guarantee optimality, but if the combinatorial decisions are not well-tamed, it can lead to exponential solution times. In fact, this has been the case presented in the literature up until now. Most exact approaches rely on integer linear programming (ILP) models [15–18].

Among the diverse taxonomy of problems within SAP, a foundational combinatorial decision lies at the core of each: given a radial power distribution network and a set of sectionalizing switches, the objective is to determine the optimal placement of these devices to maximize a predefined reliability index. We refer to this as the Sectionalizing Switch Allocation Problem (SSAP). An open question in the field has been whether efficient polynomial-time algorithms exist for the exact solution of SSAP. This work provides an answer to this question for the first time.

To the best of our knowledge, the first formal investigation of this question was conducted by Galias [19]. Galias proposed a dynamic programming (DP) algorithm capable of addressing instances with 3636 nodes and up to 15 sectionalizers. While Galias' DP algorithm exhibits worst-case exponential complexity, it paved the way for understanding that SSAP can be solved optimally in some large instances. Additionally, the author's commitment to transparency is notable, as they provided full access to their source code and network data.

* Corresponding author.

E-mail address: fusberti@ic.unicamp.br (F.L. Usberti).

Our Contributions. This paper establishes that the SSAP is polynomial-time solvable under the assumption that the network is radial and the objective function is additive (e.g., energy not supplied). This result is achieved by introducing DP-N2M2, the first exact dynamic programming algorithm with proven polynomial complexity for this class of problems. This finding has important implications for the design of efficient and scalable methods applicable to other SSAP variants. To assess the effectiveness of our approach, we conducted extensive computational experiments on 14 benchmark networks with up to 3637 nodes, comparing the performance of DP-N2M2 against the dynamic programming method proposed by Galias. In addition, we evaluated two ILP formulations for SSAP, one from Galias [19] and another from Usberti et al. [20]. These models, which had not previously been compared side by side, provide a useful baseline for assessing the performance and scalability of the proposed methodologies.

Text Organization. Section 2 provides an overview of the preliminary concepts and notations necessary to understand the SSAP and the proposed methodology. Section 3 reviews both the ILP model and DP approach proposed by Galias [19]. In Section 4, we present the ILP model of Usberti et al. [20] and outline our exact polynomial-time algorithm, DP-N2M2. Section 5 details the computational experiments, comparing our proposed methodology with previous state-of-the-art solutions. Finally, Section 6 concludes the paper.

2. Preliminary concepts

2.1. Notation

This section defines the parameters used in the methodologies proposed in this work. For the sake of clarity, decision variables will be introduced as the mathematical formulations are presented.

A radially operated distribution network is represented as an arborescence [21], i.e., a connected directed tree $T(V, A)$, with n edges and $n+1$ nodes, rooted at the substation (node 0). We denote by $T_i \subseteq T$ the subtree of T rooted at node i .

Every node other than the root, $i \in V_0$, $V_0 = V \setminus \{0\}$, is a load point of the network with an expected load l_i (kW) and an expected interruption time θ_i (hours/year). By convention, we set $l_0 = 0$ and $\theta_0 = 0$.

Each node $i \in V_0$ has a parent, denoted by $\pi(i) \in V$, and a predecessor edge $(\pi(i), i) \in A$. The children of node $i \in V$, denoted by C_i , are all nodes j such that $\pi(j) = i$. We assume an arbitrary ordering for the children of any node j , where the first child is denoted by j' , with $j' = \emptyset$ if and only if j is a leaf. Let $b(j')$ denote the next sibling of node j' , meaning $b(j')$ is the second child of j . More generally, if node j has $r+1$ children, then $C_j = \{j', b(j'), \dots, b^r(j')\}$, where $b^r(j') = b(\dots(b(j')))$ and $b^{r+1}(j') = \emptyset$. The descendants of a node $i \in V$, denoted by D_i , include all nodes (except i itself) that can be reached from node i by traversing the tree. Formally, if $C_i = \emptyset$, then $D_i = \emptyset$; otherwise, $D_i = C_i \cup \left(\bigcup_{j \in C_i} D_j\right)$.

A path $P_{ij} = (i, \dots, \pi(j), j)$ is the ordered set of nodes connecting nodes i and j . Note that P_{ij} is non-empty if and only if $j \in D_i \cup \{i\}$. The ancestors of $i \in V$, denoted by A_i , include all nodes in the path from the root to node i , i.e., $A_i = P_{0i}$.

The downstream power load $\tilde{l}_i$ and downstream interruption time $\tilde{\theta}_i$ of a node i are computed as follows:

$$\tilde{l}_i = l_i + \sum_{j \in D_i} l_j, \quad \tilde{\theta}_i = \theta_i + \sum_{j \in D_i} \theta_j$$

2.2. Switches

In this paper, we focus on sectionalizing switches, which are devices allocated to the edges of the network. For simplicity, when we say that a switch is allocated to a node i , we mean that it is allocated to the predecessor edge $(\pi(i), i)$. We assume there are m available switches, which can be allocated to a subset of nodes $V_s \subseteq V_0$.

A sectionalizing switch is a normally closed device that opens when a short circuit occurs, disconnecting all downstream load points while preventing upstream power interruptions. Since the root node has no ancestors, faults cannot propagate upstream from the root; thus, it is disregarded as a candidate for switch allocation.

Consider the network illustrated in Fig. 1, which shows three possible switch allocation scenarios. The first scenario (Fig. 1(a)) represents the worst-case reliability, where no switches are allocated, causing any fault to propagate to the root and interrupt the entire network. In the second scenario (Fig. 1(b)), a switch is placed on node 3, preventing faults at node 3 from propagating upstream. Finally, in the third scenario (Fig. 1(c)), representing the best-case reliability, a switch is installed at every node. In the following section, we will revisit this example and demonstrate how to calculate network reliability using the concept of *interruption flows*.

2.3. Reliability evaluation

The reliability of distribution systems can be assessed using several indices. The *system average interruption frequency index* (SAIFI) measures the average number of interruptions a customer experiences per year, while the *system average interruption duration index* (SAIDI) represents the total duration of interruptions for the average customer annually. Additionally, the *energy not supplied* (ENS) quantifies the total amount of energy not delivered to customers due to outages over a year [22]. Given data on expected loads and interruption durations, these reliability indices can be computed. To simplify presentation of the mathematical framework, this work focuses on one of these reliability indices as the objective function, namely the expected ENS. However, the proposed methodologies are general enough to be applied on the optimization of any of these indices.

An efficient approach to calculating a network's ENS is through interruption flows (iflows), as introduced by [20]. The iflow f_i represents the interruption time flowing backward along edge $(\pi(i), i)$, from node i to its parent $\pi(i)$. This value indicates the duration of interruptions (in hours per year) that affect $\pi(i)$ due to faults originating from i and its descendants. If a switch is placed at node i , all downstream interruptions are contained at i , resulting in an iflow f_i of zero. Mathematically, this is expressed by (1).

$$f_i = \begin{cases} 0 & i \in V_s \\ \theta_i + \sum_{j \in C_i} f_j & i \in V \setminus V_s. \end{cases} \quad (1)$$

It has been shown that ENS can be expressed in terms of iflows [20], as given by (2). The ENS expression consists of two terms. The first term captures the impact of upstream iflows — interruptions flowing toward $\pi(i)$ and affecting the load $\tilde{l}_{\pi(i)}$. The load $\tilde{l}_i$ is excluded from this term, as it is already accounted for by the iflows from i 's descendants.

$$ENS = \sum_{i \in V_0} (\tilde{l}_{\pi(i)} - \tilde{l}_i) f_i + \sum_{i \in V} \tilde{l}_i \theta_i. \quad (2)$$

The second term in (2) is a constant that provides a lower bound for the ENS. Furthermore, [20] established lower and upper bounds, E_{lb} and E_{ub} , for the ENS, as shown in (3). These bounds correspond to cases where all edges have switches (E_{lb}) and no edge has a switch (E_{ub}).

$$\sum_{i \in V} \tilde{l}_i \theta_i = E_{lb} \leq ENS \leq E_{ub} = \tilde{l}_0 \tilde{\theta}_0 \quad (3)$$

The networks depicted in Fig. 1 can have their ENS evaluated using (2) and (3).

$$ENS_{\text{(Fig. 1(a))}} = E_{ub} = \tilde{l}_0 \tilde{\theta}_0 = 100.0 \text{ kWh/yr},$$

$$\begin{aligned} ENS_{\text{(Fig. 1(b))}} &= (\tilde{l}_0 - \tilde{l}_1) f_1 + (\tilde{l}_1 - \tilde{l}_2) f_2 + (\tilde{l}_1 - \tilde{l}_3) f_3 + E_{lb} \\ &= 70 \text{ kWh/yr}, \end{aligned}$$

$$ENS_{\text{(Fig. 1(c))}} = E_{lb} = \tilde{l}_0 \theta_0 + \tilde{l}_1 \theta_1 + \tilde{l}_2 \theta_2 + \tilde{l}_3 \theta_3 = 50.5 \text{ kWh/yr}.$$

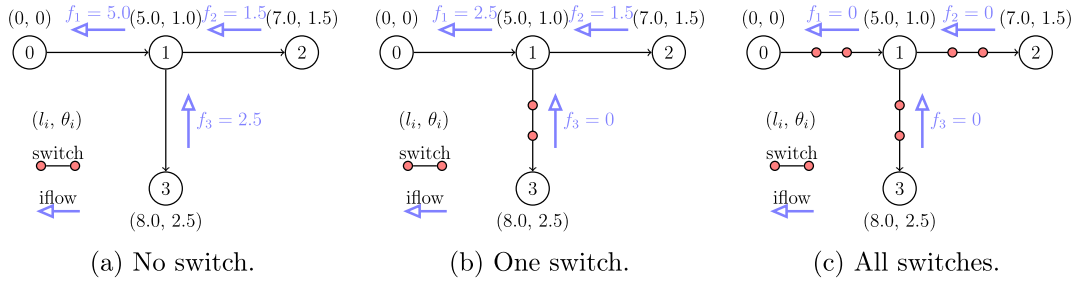


Fig. 1. Small examples of switch allocations.

2.4. Problem definition

The combinatorial optimization problem of allocating sectionalizing switches in a radial distribution network is formally defined as follows:

Definition: Sectionalizing Switch Allocation Problem (SSAP)

Input: An arborescence $T(V, A)$ with non-negative load l_i and interruption time θ_i for each node $i \in V$, and a constant m , representing the number of available sectionalizing switches.

Solution: Determine an optimal subset of m nodes $V_s \subseteq V_0$ for switch allocation, minimizing the ENS.

3. Previous work

This section highlights the work of Galias [19], which represents the state-of-the-art in solving the SSAP. Galias proposes both an Integer Linear Programming (ILP) model and a tree search-based algorithm for addressing the SSAP, each briefly described below.

3.1. Mathematical model

The ILP model for the SSAP, proposed by Galias [19], is outlined as follows. The binary decision variables x_i indicate whether a switch is allocated to node i ($x_i = 1$) or not ($x_i = 0$). Additionally, the binary auxiliary variables z_{ij} determine whether the path between two switches located at nodes i and j contains any other switch ($z_{ij} = 0$) or not ($z_{ij} = 1$).

The objective function (4) represents the ENS of the solution. Its first term provides an upper bound on the ENS, from which the second term subtracts the effect of each allocated switch as if it were the only switch in the network. The third and final term compensates for the second term by accounting for pairs of switches with no other switch along their path. The number of switches is constrained by (5). Constraints (6) and (7) set the appropriate values of z_{ij} , where $z_{ij} = 1$ if and only if $x_i = x_j = 1$ and $x_k = 0$ for every $k \in \mathcal{P}_{ij} \setminus \{i, j\}$. The number of variables and constraints in the ILP model proposed by Galias are both bounded by $O(n^2)$.

(ILP-GALIAS)

$$\begin{aligned} \text{MIN } \text{ENS} = & E_{ub} - \sum_{i \in V_0} (\tilde{l}_0 - \tilde{l}_i) \tilde{\theta}_i x_i \\ & + \sum_{i \in V_0} \sum_{j \in D_i} (\tilde{l}_0 - \tilde{l}_i) \tilde{\theta}_j z_{ij} \end{aligned} \quad (4)$$

s.t.

(number of switches)

$$\sum_{i \in V_0} x_i = m \quad (5)$$

(interswitch-path constraints)

$$x_i + x_j + \sum_{k \in \mathcal{P}_{ij} \setminus \{i, j\}} (1 - x_k) \geq |\mathcal{P}_{ij}| z_{ij} \quad i \in V_0, j \in D_i \quad (6)$$

$$x_i + x_j + \sum_{k \in \mathcal{P}_{ij} \setminus \{i, j\}} (1 - x_k) \leq |\mathcal{P}_{ij}| + z_{ij} - 1 \quad i \in V_0, j \in D_i \quad (7)$$

(variables integrality)

$$x_i, z_{ij} \in \{0, 1\} \quad i \in V_0, j \in D_i \quad (8)$$

3.2. Tree Search (TS) based algorithm

Galias [19] proposes a dynamic programming approach, referred to herein as DP-TS, in which partial solutions are determined recursively. The algorithm starts from load points and progresses upward through the tree. For each node, partial solutions are generated based on the partial solutions of its children. Once the root is reached, solutions for the entire network are obtained, and one that minimizes ENS is selected.

Since DP-TS generates all possible partial solutions for a fixed number of switches, it encounters exponential computational complexity. To alleviate this, Galias introduces a procedure to eliminate certain partial solutions. Specifically, conditions are identified where a partial solution at a given node is dominated by another partial solution. Once a dominated solution is identified, it cannot contribute to an optimal solution for the entire tree and can therefore be safely discarded.

Galias empirically demonstrated that filtering partial solutions based on these dominance conditions significantly reduces the search space. However, this approach does not eliminate the algorithm's worst-case exponential complexity. Consequently, solving the SSAP remains intractable for DP-TS in instances where the dominance conditions are less effective. In Section 5, we present examples of such cases.

4. Our work

4.1. Mathematical model

Using the concept of iflows, the integer linear programming model ILP-IFLOWS for the SSAP, proposed by [20], is presented here. In this work, ILP-IFLOWS is also compared with other approaches for the first time. In the following formulation, the binary decision variables x_i indicate whether a switch is allocated at node i ($x_i = 1$) or not ($x_i = 0$).

The objective function (9) minimizes the solution ENS, calculated as in (2). The number of switches is limited by constraint (10). Constraints (11) set the values of the iflows, where M_i is a sufficiently large constant. These constraints model the definition of iflows as given by (1). If $x_i = 1$, the constraint becomes inactive, allowing f_i to be zero. Conversely, if $x_i = 0$, constraints (11) enforce $f_i \geq \theta_j + \sum_{j \in C_i} f_j$. Since the objective function minimizes the iflows, this constraint will hold as an equality in the optimal solution.

The number of variables and constraints in ILP-IFLOWS are both bounded by $O(n)$, marking a significant improvement in model size compared to ILP-GALIAS [19]. It is also worth noting that ILP-IFLOWS is a slight improvement over its original model in [20]. For a radial network with n nodes, ILP-IFLOWS introduces only two variables per node, one binary x_i and one continuous iflow f_i , for a total of $2n$ variables,

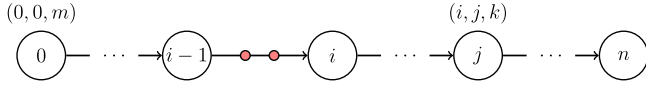


Fig. 2. Linear network with states $(0, 0, m)$ and (i, j, k) .

together with n linking constraints and a single budget equation ($n + 1$ constraints overall). The earlier formulation [20], by contrast, keeps a binary and a flow variable on every line and adds one extra continuous variable per node, reaching roughly $3n$ variables and $2n + 1$ constraints. Hence ILP-IFlows reduces both the variable count and the constraint count by about one-third, resulting in a leaner mixed-integer model.

(ILP-IFlows)

$$\text{MIN } ENS = \sum_{i \in V_0} (\tilde{l}_{\pi(i)} - \tilde{l}_i) f_i + E_{lb} \quad (9)$$

s.t.

(number of switches)

$$\sum_{i \in V_0} x_i = m \quad (10)$$

(iflows constraints)

$$f_i \geq \theta_i + \sum_{j \in C_i} f_j - M_i x_i \quad i \in V_0 \quad (11)$$

(variables non-negativity and integrality)

$$f_i \geq 0 \quad i \in V_0 \quad (12)$$

$$x_i \in \{0, 1\} \quad i \in V_0 \quad (13)$$

4.2. Polynomial-time dynamic programming

In this section, we introduce DP-N2M2, the first polynomial-time exact algorithm for the SSAP. It is based on dynamic programming (DP), a methodology to solve complex optimization problems that exhibit optimal substructure and overlapping subproblems. This paradigm has proven especially effective in power system applications, where problems often involve decisions over tree-like structures. Notable examples of DP in power systems include sensor placement for fault detection [23], optimal capacitor placement [24], and service restoration [25].

In our setting, we defined the state of the DP as a triple (i, j, k) , where $j \in V$, $i \in A_j$, and k is a non-negative integer representing the number of switches, with $0 \leq k \leq m$.

We define two DP optimality functions: one for linear networks (Section 4.2.1) and another for branched networks (Section 4.2.2).

4.2.1. Linear network

In this case, we consider networks where each node has at most one child. For these linear networks, we can assume that nodes are sequentially numbered from the root node 0 to the leaf node n (Fig. 2).

Let the state (i, j, k) be interpreted as follows: j is the current node under consideration for switch allocation, i is the closest ancestor of j that contains a switch, and k is the number of remaining switches available for allocation. If no ancestral node has a switch, then i points to the root.

We define the optimality function $ENS(i, j, k)$ as a recursive function that provides the minimum ENS in the subtree T_i , accounting for faults originating in subtree T_j and the optimal allocation of k switches in T_j .

DP-N2M2 employs a recursive top-down approach, starting from the root and solving the problem by recursively descending through its subtrees (subproblems) until reaching a base case. Once a subtree is solved, its corresponding optimal solution is memoized by mapping the

state (i, j, k) to its optimal value. This memoization of solved states is crucial to avoid combinatorial explosion.

Before presenting the mathematical description of $ENS(i, j, k)$, we first discuss three base cases, followed by the general case.

Case 1. Root. The first state evaluated by the DP, $(0, 0, m)$, starts at the root. Here, $ENS(0, 0, m)$ provides the minimum ENS for the entire tree $T_0 = T$ when allocating m switches in T_0 . This corresponds to the definition of the optimal solution for the SSAP. However, since placing a switch at the substation is unnecessary, this state can be skipped in favor of starting at node 1, as shown below:

$$ENS(0, 0, m) = ENS(0, 1, m) \quad (14)$$

Case 2. No more switches. Suppose the state (i, j, k) is such that $k = 0$. In this case, the DP optimality function simplifies to:

$$ENS(i, j, k) = \tilde{l}_i \tilde{\theta}_j \quad k = 0 \quad (15)$$

Eq. (15) indicates that once all available switches are exhausted, the DP search can terminate, and the resulting ENS should reflect the downstream interruption time $\tilde{\theta}_j$ affecting the power load of all nodes in the subtree T_i .

Case 3. Leaf node. Now consider the DP state (i, j, k) at the leaf node n , where $j = n$ and $k > 0$. In this case, the DP optimality function is evaluated as follows:

$$ENS(i, j, k) = l_j \theta_j \quad j = n, k > 0 \quad (16)$$

Eq. (16) shows that if we reach the leaf with remaining switches, a switch should be allocated at node n , causing the interruption time θ_n to only affect the load l_n .

General case. In the general case, we are in a state (i, j, k) where we have not yet reached a leaf node ($j < n$) and at least one switch remains available ($k > 0$). For this situation, there are two possible decisions: to allocate a switch at node j or not. Eq. (17) describes the effects on ENS for these two decisions, retaining the best choice (minimum ENS).

$$ENS(i, j, k) = \min \left\{ \underbrace{ENS(i, j+1, k) + \tilde{l}_i \theta_j}_{\text{no switch at } j}, \underbrace{ENS(j, j+1, k-1) + \tilde{l}_j \theta_j}_{\text{switch at } j} \right\} \quad j < n, k > 0 \quad (17)$$

If we choose not to allocate a switch, the state transition leads from (i, j, k) to $(i, j+1, k)$. This means that, while moving to the next node $j+1$, the closest ancestor switch i and the number of available switches k remain unchanged. The ENS increment term $\tilde{l}_i \theta_j$ indicates that the interruption time θ_j will propagate to the downstream load $\tilde{l}_i$ of the closest ancestor switch.

Alternatively, allocating a switch at node j leads to the state $(j, j+1, k-1)$. In this transition, the closest ancestor switch is updated to node j , and the number of available switches decreases by one. The ENS is incremented by $\tilde{l}_j \theta_j$.

The complexity of the DP for linear networks is given by the number of states $O(n^2 m)$ times the effort required to solve each state $O(1)$. Thus, the linear SSAP can be solved in $O(n^2 m)$ in the worst case.

4.2.2. Branched network

This section addresses branched networks, where some nodes may have multiple children. The state (i, j, k) and the DP optimality function $ENS(i, j, k)$ retain the same semantics as in the linear case. However, special consideration is needed when a branched node j has $r + 1$ children ($r > 0$), such that $C_j = \{j', b(j'), \dots, b^r(j')\}$, as shown in Fig. 3.

For each network branch, the DP must determine how many switches should be allocated to each branch. To this end, we define an auxiliary optimality function $\overline{ENS}(i, j', k)$, which captures the minimum ENS in subtree $T_{j'}$, accounting for faults originating in the sibling subtrees $T_{j'}, T_{b(j')}, \dots, T_{b^r(j')}$, and the optimal allocation of k switches

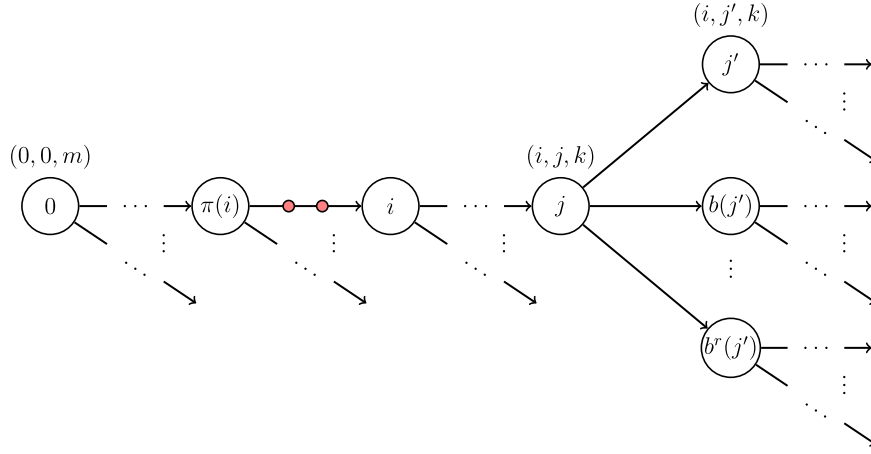


Fig. 3. Branched network illustrating states $(0, 0, m)$, (i, j, k) and (i, j', k) .

Table 1

Network characteristics.

Dataset	Network	Edges	Load (kW)	Customers	Feeders
REDS ^a	R3	33	3 708.27	1 816	1
	R4	94	28 342.96	14 076	11
	R5	143	18 315.82	7 987	8
	R6	204	27 571.37	12 975	3
	R7	880	124 920.01	59 698	7
Poland ^b	pr76zb2	77	3 074.00	1 484	1
	pr112zb2	113	4 707.00	2 337	1
	pr199zb2	200	9 838.00	4 060	1
	pr262zb2	263	13 193.00	5 705	1
IEEE ^c	n384	384	1 647.57	201	1
	n568	568	2 498.01	301	1
	n1149	1149	4 904.38	601	1
	n2250	2250	10 773.17	1 177	1
	n3637	3637	10 773.17	1 177	1

^a Repository of Distribution Systems.

^b Poland networks.

^c IEEE networks

in these same sibling subtrees, where $b'(j')$ is the last sibling of j' . This definition allows us to generalize the linear DP optimality function for any state (i, j, k) , even if j has multiple children (18).

$$ENS(i, j, k) = \quad (18)$$

$$\begin{cases} \bar{l}_i \bar{\theta}_j & k = 0 \\ l_j \theta_j & j' = \emptyset, k > 0 \\ \min[\underbrace{ENS(i, j', k) + \bar{l}_i \bar{\theta}_j}_{\text{no switch at } j}, \underbrace{ENS(j, j', k-1) + \bar{l}_j \theta_j}_{\text{switch at } j}] & \text{otherwise} \end{cases}$$

The auxiliary DP function $\overline{ENS}(i, j', k)$ determines the optimal distribution of switches between the j' branch and any remaining sibling branches. It is formulated in (19):

$$\overline{ENS}(i, j', k) = \quad (19)$$

$$\begin{cases} ENS(i, j', k) & b(j') = \emptyset \\ \min_{0 \leq q \leq k} [ENS(i, j', q) + \overline{ENS}(i, b(j'), k-q)] & b(j') \neq \emptyset \end{cases}$$

j' branch j' remaining siblings

Eq. (19) shows that if the DP state is at a node j' with no remaining siblings ($b(j') = \emptyset$), then there is no need to distribute switches any further, just give all remaining k switches to branch j' . However, if siblings remain, the DP searches for the optimal distribution of switches between branch j' and the subsequent branches, considering every possible value q ($0 \leq q \leq k$) for the allocation of switches to branch j' .

The DP for branched networks has $O(n^2m)$ states, as in the linear case. The computation required for each state is dominated by the auxiliary DP function $\overline{ENS}(i, j', k)$, which checks $O(m)$ possible switch allocations per branch. Consequently, the SSAP for branched networks can be solved in $O(n^2m^2)$ in the worst case.

4.3. Pseudocodes

Algorithm 1 implements the recursive evaluation of $ENS(i, j, k)$, considering whether to place a switch at node j or not, and recursively evaluating the associated costs. To support this, Algorithm 2 computes $\overline{ENS}(i, j', k)$, which aggregates the optimal ENS across a set of sibling subtrees starting from branch j' . These two algorithms operate jointly within the dynamic programming framework, making use of memoization to efficiently explore all feasible switch allocation states.

Algorithm 1: EVALUATION OF $ENS(i, j, k)$

Input: State (i, j, k) , such that j is the current node under consideration, i is the closest ancestor of j that contains a switch, and k is the number of remaining switches available for allocation.

Output: The value of $ENS(i, j, k)$, which is the minimum ENS in the subtree T_j , accounting for faults originating in subtree T_j and the optimal allocation of k switches in T_j .

Note: $memo(i, j, k)$ is a memoization table, declared as a global variable. It stores the value associated with the state (i, j, k) (illustrated in Fig. 3) and is initialized to ∞ to indicate that the state has not yet been solved.

```

begin
  if  $memo(i, j, k) = \infty$  then
    // The following cases implement Eq. (18)
    if  $k = 0$  then
      |  $memo(i, j, k) \leftarrow \bar{l}_i \bar{\theta}_j$ 
    else if  $j' = \emptyset$  then
      |  $memo(i, j, k) \leftarrow l_j \theta_j$ 
    else
      |  $memo(i, j, k) \leftarrow \min[\overline{ENS}(i, j', k) + \bar{l}_i \bar{\theta}_j, \overline{ENS}(j, j', k-1) + \bar{l}_j \theta_j]$ 
    return  $memo(i, j, k)$ 
end

```

4.4. Comparison between DP-TS and DP-N2M2

Although both algorithms solve the same underlying optimization problem using dynamic programming, DP-TS and DP-N2M2 differ significantly in their design principles, state definitions, and computational complexity.

Table 2
Average results per network.

REDS	ILP-GALIAS		ILP-IFLOWS		DP-TS		DP-N2M2	
	sol (%)	Time	sol (%)	Time	sol (%)	Time	sol (%)	Time
R3	all	3 494.05	all	2.14	all	<0.01	all	<0.01
R4	16.13	<i>tle</i>	all	202.68	9.68	<i>tle</i>	all	<0.01
R5	7.75	<i>tle</i>	28.87	<i>tle</i>	7.04	<i>tle</i>	all	<0.01
R6	2.96	<i>tle</i>	17.73	<i>tle</i>	all	518.00	all	0.03
R7	0.80	<i>tle</i>	1.93	<i>tle</i>	0.80	<i>tle</i>	all	1.26
Average	5.39		16.40		19.65		all	
Poland	ILP-GALIAS		ILP-IFLOWS		DP-TS		DP-N2M2	
	sol (%)	Time	sol (%)	Time	sol (%)	Time	sol (%)	Time
pr76zb2	19.74	<i>tle</i>	all	53.74	all	0.30	all	0.01
pr112zb2	9.82	<i>tle</i>	71.43	<i>tle</i>	all	1.16	all	0.02
pr199zb2	4.52	<i>tle</i>	26.13	<i>tle</i>	all	5 495.66	all	0.05
pr262zb2	3.44	<i>tle</i>	18.32	<i>tle</i>	12.60	<i>tle</i>	all	0.08
Average	7.04		39.82		64.93		all	
IEEE	ILP-GALIAS		ILP-IFLOWS		DP-TS		DP-N2M2	
	sol (%)	Time	sol (%)	Time	sol (%)	Time	sol (%)	Time
n384	1.57	<i>tle</i>	6.27	<i>tle</i>	all	106.18	all	1.69
n568	0.88	<i>tle</i>	4.06	<i>tle</i>	all	559.50	all	11.12
n1149	0.35	<i>tle</i>	1.48	<i>tle</i>	all	6 091.76	all	89.51
n2250	0.09	<i>tle</i>	0.58	<i>tle</i>	12.85	<i>tle</i>	all	847.67
n3637	0.06	<i>tle</i>	0.17	<i>tle</i>	5.23	<i>tle</i>	all	5 857.92
Average	0.25		1.09		32.32		all	

sol (%): percentage of solved instances. time: execution time in seconds.

all: all instances solved. *tle*: time limit exceeded.

Algorithm 2: EVALUATION OF $\overline{ENS}(i, j', k)$

Input: State (i, j', k) , such that j' is the current branch under consideration, i is the closest ancestor of j' that contains a switch, and k is the number of remaining switches available for allocation.

Output: The value of $\overline{ENS}(i, j', k)$, which is the minimum ENS in subtree T_i , accounting for faults originating in the sibling subtrees $T_{j'}$, $T_{b(j')}$, ..., $T_{b(j', k)}$, and the optimal allocation of k switches in these same sibling subtrees.

Note: $\overline{memo}(i, j', k)$ is a memoization table, declared as a global variable. It stores the value associated with the state (i, j', k) (illustrated in Fig. 3) and is initialized to ∞ to indicate that the state has not yet been solved.

```

begin
  if  $\overline{memo}(i, j', k) = \infty$  then
    // The following cases implement Eq. (19)
    if  $b(j') = \emptyset$  then
      |  $\overline{memo}(i, j', k) \leftarrow \overline{ENS}(i, j', k)$ 
    else
      for  $q \leftarrow 0$  to  $k$  do
        |  $\overline{memo}(i, j', k) \leftarrow \min[\overline{memo}(i, j', k), \overline{ENS}(i, j', q) + \overline{ENS}(i, b(j'), k - q)]$ 
      return  $\overline{memo}(i, j', k)$ 
end

```

A first and fundamental distinction lies in the direction of the traversal. DP-TS builds solutions bottom-up, combining sets of partial solutions from child subtrees at each internal node. In contrast, DP-N2M2 proceeds top-down from the root toward the leaves, which enables a compact state definition and simplifies the recursive structure of the algorithm, as the interruption effect and switch budget are fully known throughout the descent. The bottom-up traversal used in DP-TS cannot anticipate these quantities in advance and must therefore maintain large sets of partial solutions to preserve all feasible combinations of local decisions.

This difference in strategy has important consequences for the size and structure of the state space. In DP-TS, each node must enumerate all combinations of partial solutions from its children, leading to an exponential number of records in cases where the branching factor or the switch budget is large. Even with dominance filtering, the number

of subproblem configurations can grow rapidly. On the other hand, DP-N2M2 encodes each subproblem as a single memoised state (i, j, k) . The algorithm considers only two transitions per state, either keeping the current ancestor or placing a switch at j , and delegates the distribution of switches among child branches to an efficient sibling-allocation subroutine that runs in $O(m)$ time.

As a result, the overall complexity of DP-TS is exponential in the worst case, bounded by $\sum_j (m+1)^{|C_j|}$, where $|C_j|$ denotes the number of children of node j . In contrast, DP-N2M2 solves at most $O(n^2 m)$ distinct states and achieves time and space bounds of $O(n^2 m^2)$ and $O(n^2 m)$, respectively. The top-down design not only avoids redundant recomputation and combinatorial explosion but also leads to reduced memory overhead in practice, as confirmed by the experimental results in Section 5.

5. Computational experiments

5.1. Computational environment and benchmark

In this section, we compare the performance of four methodologies: two mathematical models (ILP-GALIAS and ILP-IFLOWS) and two dynamic programming algorithms (DP-TS and DP-N2M2). An instance is defined as a pair consisting of a distribution network and a specified number of available switches. We considered 14 networks from three datasets (Table 1), and for each network, we evaluated all possible numbers of switches, from $m = 0$ up to $m = n$. This results in a benchmark comprising a total of 9995 instances. Each methodology was allocated two hours per network or one hour per instance, terminating if either limit was reached. The full results and source code for these computational experiments are publicly available.¹ All methodologies were implemented by the authors, with the exception of DP-TS, for which the original source code was made available by Galias [19].²

The experiments were conducted in the following computational environment: Ubuntu 22.04, with a 64-bit Intel 11900H processor and 16 GB RAM. The mathematical models were solved using Gurobi 11.0.2.

¹ www.ic.unicamp.br/~fusberty/problems/ssap.

² www.zet.agh.edu.pl/ncn2015power/.

Table 3
Results for selected instances of REDS dataset.

Instance		ENS	Time (s)			
Network	<i>m</i>		ILP-GALIAS	ILP-IFLOWS	DP-TS	DP-N2M2
R3	0	11 135.20	<0.01	0.01	<0.01	<0.01
	10	2 372.86	13.73	0.04	0.01	<0.01
	20	2 132.20	190.74	0.18	<0.01	<0.01
	30	2 071.62	126.24	<0.01	<0.01	<0.01
	32	2 069.97	165.92	<0.01	<0.01	<0.01
R4	0	42 302.40	<0.01	<0.01	<0.01	<0.01
	9	7 697.22	0.19	<0.01	<u>2 582.70</u>	<0.01
	10	5 968.14	0.35	<0.01	–	<0.01
	<u>14</u>	3 130.15	<u>817.89</u>	0.04	–	<0.01
	20	2 725.95	–	0.14	–	<0.01
	30	2 489.07	–	0.72	–	<0.01
	40	2 398.36	–	11.65	–	<0.01
	50	2 355.54	–	0.99	–	<0.01
	93	2 340.32	–	<0.01	–	<0.01
R5	0	107 230.00	<0.01	<0.01	<0.01	<0.01
	<u>10</u>	11 096.70	<u>1 431.28</u>	0.04	<u>1 992.82</u>	<0.01
	20	7 025.91	–	12.53	–	<0.01
	30	5 371.94	–	5.99	–	<0.01
	40	4 558.26	–	1 345.13	–	<0.01
	<u>41</u>	4 504.61	–	<u>2 402.80</u>	–	<0.01
	50	4 177.97	–	–	–	<0.01
	100	3 759.45	–	–	–	<0.01
	142	3 747.42	–	–	–	<0.01
R6	0	20 806.50	0.01	<0.01	<0.01	<0.01
	5	5 864.04	<u>293.23</u>	0.11	0.03	<0.01
	10	4 075.70	–	0.12	0.16	<0.01
	30	1 959.61	–	261.62	2.73	<0.01
	<u>35</u>	1 817.08	–	<u>977.70</u>	3.45	<0.01
	50	1 577.59	–	–	5.25	<0.01
	100	1 447.70	–	–	4.42	<0.01
	150	1 437.63	–	–	0.58	<0.01
R7	0	8 974 232.50	0.15	<0.01	<0.01	<0.01
	<u>7</u>	1 518 308.94	<u>2 772.48</u>	0.06	<u>2 121.59</u>	<0.01
	16	885 747.91	–	<u>1 667.82</u>	–	0.01
	50	520 262.58	–	–	–	0.01
	100	378 224.25	–	–	–	0.01
	300	280 298.17	–	–	–	<0.01
	500	268 834.04	–	–	–	<0.01
	700	266 590.14	–	–	–	<0.01
	879	266 293.63	–	–	–	<0.01

Underline: highest number of switches solved by a methodology.

5.2. Overall results

Fig. 4 shows the performance profiles for the methodologies. These profiles display the rate of solved instances as a function of the time allowed per instance for each methodology. A clear dominance relation is evident among the methodologies. Specifically, at any given time frame, the methodologies can be ordered by decreasing number of solved instances as follows: DP-N2M2, DP-TS, ILP-IFLOWS, and ILP-GALIAS. Since the curves are presented on a log-log scale, the gaps between them indicate substantial improvements in the rate of solved instances.

With one hour of execution time per instance, ILP-GALIAS was able to solve 1.38% of the instances. The same result was achieved by ILP-IFLOWS in less than 0.04 s. After one hour, ILP-IFLOWS solved 5.69% of the instances, showing that, while it performs better than ILP-GALIAS, it still suffers from combinatorial explosion.

The DP-TS algorithm solved 32.74% of the instances within the one-hour time limit, taking only 0.21 s to exceed the 5.69% solved by the best ILP model in one hour. These results confirm the effectiveness of DP-TS in reducing the search space and mitigating combinatorial explosion for many instances that were intractable for the ILP models. Nonetheless, a significant portion of instances remained too complex for DP-TS to reach optimality within the imposed time limits.

In contrast, DP-N2M2 was able to solve all instances, taking no more than 7.72 s in the worst case. Within the same time frame, DP-TS solved

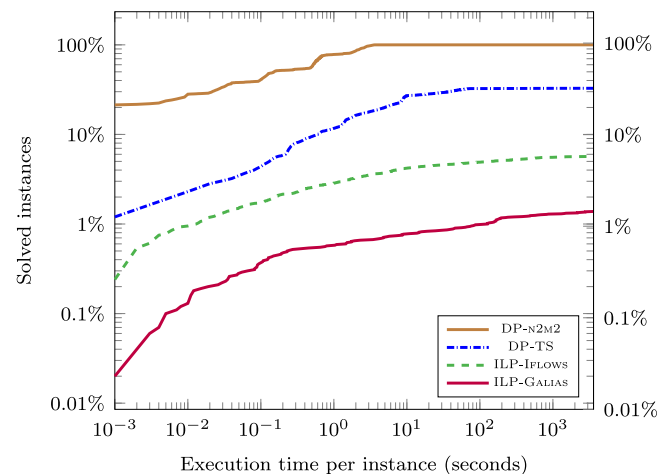


Fig. 4. Performance profile of all methodologies.

22.40% of the instances. Moreover, DP-N2M2 took less than 0.03 s to solve the same number of instances that DP-TS achieved in one hour.

Table 4
Results for selected instances of Poland dataset.

Instance		ENS	Time (s)			
Network	<i>m</i>		ILP-GALIAS	ILP-IFLOWS	DP-TS	DP-N2M2
pr76zb2	0	7 202.21	<0.01	<0.01	<0.01	<0.01
	10	1 561.84	84.48	0.04	<0.01	<0.01
	<u>15</u>	1 296.60	<u>2 457.14</u>	0.08	0.01	<0.01
	20	1 114.37	–	0.30	0.01	<0.01
	30	885.78	–	0.54	0.01	<0.01
	40	775.61	–	1.64	0.01	<0.01
	50	715.50	–	1.99	<0.01	<0.01
	76	675.89	–	<0.01	<0.01	<0.01
pr112zb2	0	15 958.20	0.003	<0.01	<0.01	<0.01
	<u>10</u>	3 897.37	<u>2 176.14</u>	0.15	0.01	<0.01
	20	2 773.64	–	0.99	0.02	<0.01
	30	2 182.86	–	3.11	0.02	<0.01
	40	1 821.80	–	7.06	0.01	<0.01
	50	1 589.53	–	3.68	0.02	<0.01
	<u>80</u>	1 359.85	–	<u>375.88</u>	0.01	<0.01
	100	1 330.97	–	–	<0.01	<0.01
	112	1 326.86	–	–	<0.01	<0.01
pr199zb2	0	58 226.30	0.01	<0.01	<0.01	<0.01
	<u>8</u>	11 682.60	<u>2 168.20</u>	0.06	0.11	<0.01
	10	10 063.70	–	0.18	0.26	<0.01
	20	6 693.08	–	1.68	2.92	<0.01
	30	5 277.99	–	6.28	9.38	<0.01
	40	4 420.82	–	69.52	18.39	<0.01
	50	3 863.21	–	1 282.21	29.21	<0.01
	<u>52</u>	3 770.89	–	<u>1 231.47</u>	31.56	<0.01
	100	2 677.70	–	–	65.65	<0.01
	150	2 408.98	–	–	19.62	<0.01
	199	2 367.81	–	–	0.09	<0.01
pr262zb2	0	105 636.00	0.01	<0.01	<0.01	<0.01
	<u>8</u>	19 691.10	<u>2 783.66</u>	0.03	1.13	<0.01
	10	17 286.40	–	0.06	3.99	<0.01
	20	11 010.80	–	1.29	120.66	<0.01
	30	8 737.34	–	29.36	622.88	<0.01
	<u>33</u>	8 234.66	–	75.78	<u>891.76</u>	<0.01
	40	7 394.01	–	342.54	–	<0.01
	<u>48</u>	6 601.72	–	<u>1 033.72</u>	–	<0.01
	50	6 432.62	–	–	–	<0.01
	100	4 276.05	–	–	–	<0.01
	200	3 422.75	–	–	–	<0.01
	262	3 369.75	–	–	–	<0.01

Underline: highest number of switches solved by a methodology.

5.3. Results per network

Table 2 presents the performance of each methodology across different networks. The table shows the percentage of instances solved by each methodology within the given time limits for each network. A clear trend emerges concerning network size: as network size increases, the percentage of solved instances decreases, and the time required to solve instances increases. In fact, the only network solved by all methods was R3, the smallest network. An exception to this trend is observed with DP-TS, which, despite solving all instances of R6, solved less than 10% of instances for R4 and approximately 7% for R5. This suggests that the complexity of DP-TS may depend not only on network size but also on network topology.

As shown in Table 2, the Poland networks were the easiest for all methodologies, as they yielded the highest percentage of solved instances. Conversely, the IEEE dataset was the most challenging for both models and for DP-N2M2, with both models achieving a small percentage of solved instances and DP-N2M2 requiring more than an hour and a half of execution time on this dataset. On the other hand, the REDS dataset posed the greatest difficulty for DP-TS. This difficulty may be related to the distribution of loads and failures within the networks in this dataset, which may hinder the elimination of dominated partial solutions in DP-TS.

5.4. Results per instance

An excerpt of the numerical results is presented for a subset of instances from the REDS (Table 3), Poland (Table 4), and IEEE (Table 5) datasets. The entire set of instances from the REDS dataset was solved quickly by DP-N2M2, taking two-hundredths of a second or less per instance. In contrast, DP-TS struggled with many instances in this set and was unable to progress beyond 10 switches for three networks. The ILP-IFLOWS model outperformed ILP-GALIAS on every network and, in some cases, also outperformed DP-TS. For example, on the largest network, R7, ILP-IFLOWS obtained optimal solutions for instances up to 16 switches, whereas DP-TS halted after seven switches.

The Poland dataset was also efficiently solved by DP-N2M2, which took less than one-hundredth of a second per instance. DP-TS solved all instances in this dataset except for the largest network, pr262zb2, where it managed to solve instances up to 33 switches. While ILP-IFLOWS was generally outperformed by DP-TS, it competed favorably on the largest instance, solving up to 48 switches.

The IEEE dataset proved the most challenging in terms of the number of solved instances and computational times, primarily due to the large network sizes (on the order of thousands of nodes). DP-N2M2 was able to achieve optimality for all instances, though the larger networks demanded increasing computational times. The n3637 network was the only one requiring, on average, more than one second of execution time per instance for DP-N2M2. For this dataset, DP-TS

Table 5
Results for selected instances of IEEE dataset.

Instance		ENS	Time (s)			
Network	m		ILP-GALIAS	ILP-IFLOWS	DP-TS	DP-N2M2
n384	0	3 388.78	0.09	<0.01	<0.01	<0.01
	<u>5</u>	1 389.79	<u>483.84</u>	0.10	0.06	<0.01
	<u>24</u>	693.59	–	<u>2 976.77</u>	0.34	<0.01
	50	534.32	–	–	0.58	<0.01
	100	475.89	–	–	0.65	0.01
	200	452.36	–	–	0.23	<0.01
	300	447.99	–	–	0.10	<0.01
	383	446.96	–	–	0.01	<0.01
n568	0	8 812.34	0.249	0.001	<0.01	<0.01
	<u>4</u>	3 610.72	<u>1 508.63</u>	0.52	0.09	<0.01
	<u>23</u>	1 479.98	–	<u>2 247.90</u>	0.66	0.02
	30	1 328.04	–	–	0.83	0.01
	50	1 094.36	–	–	1.11	0.02
	100	923.62	–	–	1.64	0.04
	200	851.73	–	–	1.43	0.04
	300	834.95	–	–	1.33	0.02
	400	829.59	–	–	0.56	<0.01
	500	827.37	–	–	0.13	<0.01
	567	826.88	–	–	0.02	<0.01
n1149	0	30 093.20	1.51	<0.01	<0.01	<0.01
	<u>3</u>	13 459.60	<u>15.93</u>	0.35	0.11	0.02
	<u>16</u>	5 712.89	–	<u>413.64</u>	1.02	0.08
	50	3 495.80	–	–	3.67	0.09
	100	2 780.18	–	–	5.45	0.13
	200	2 415.05	–	–	9.07	0.13
	300	2 303.60	–	–	8.71	0.15
	400	2 259.88	–	–	9.19	0.16
	500	2 236.88	–	–	9.12	0.10
	1 000	2 207.63	–	–	1.00	0.03
	1 148	2 206.52	–	–	0.11	<0.01
n2250	0	98 575.40	8.93	<0.01	0.02	<0.01
	<u>2</u>	51 488.40	<u>3 553.16</u>	0.06	0.14	0.08
	<u>13</u>	19 888.90	–	<u>3 238.71</u>	1.92	0.15
	50	10 335.40	–	–	8.79	0.31
	100	7 450.50	–	–	16.47	0.45
	200	6 064.34	–	–	35.59	0.47
	<u>289</u>	5 628.63	–	–	<u>50.49</u>	0.49
	300	5 592.95	–	–	–	0.51
	400	5 380.51	–	–	–	0.53
	500	5 260.07	–	–	–	0.50
	1 000	5 069.20	–	–	–	0.78
	2 000	5 017.30	–	–	–	0.12
	2 249	5 015.63	–	–	–	<0.01
n3637	0	98 575.26	66.62	0.01	0.05	<0.01
	<u>1</u>	73 457.10	<u>617.25</u>	0.25	0.11	<0.01
	<u>6</u>	30 418.69	–	<u>2 965.20</u>	0.92	0.31
	50	10 335.38	–	–	18.57	0.90
	100	7 450.47	–	–	41.78	1.10
	<u>188</u>	6 153.33	–	–	<u>71.95</u>	1.82
	200	6 064.32	–	–	–	1.86
	300	5 592.92	–	–	–	1.66
	400	5 380.48	–	–	–	1.89
	500	5 260.05	–	–	–	1.92
	1 000	5 069.17	–	–	–	2.27
	2 000	5 017.28	–	–	–	2.21
	3 636	5 015.60	–	–	–	<0.01

Underline: highest number of switches solved by a methodology.

could not progress beyond 289 switches for the n2250 network and 188 switches for the n3637 network. The optimal solution for network n3637 with $m = 100$ is shown in Fig. 5. The ILP models struggled with this dataset: ILP-GALIAS was unable to solve instances beyond five switches, while ILP-IFLOWS could not surpass 24 switches.

6. Final remarks

This work addresses a longstanding open question in the field of distribution network reliability optimization by presenting the first exact polynomial-time algorithm for the Sectionalizing Switch Allocation Problem (SSAP). Through a rigorous exploration of dynamic

programming strategies, we developed and demonstrated the DP-N2M2 algorithm, which not only solves SSAP efficiently but also establishes that the problem is tractable, thereby refuting prior assumptions about its computational complexity.

Our methodology, benchmarked against state-of-the-art approaches, including ILP models and previous dynamic programming techniques, showed substantial improvements. In particular, DP-N2M2 achieved optimal solutions with significantly reduced computational times, even for large networks. This was evident in our comprehensive experiments, encompassing 9995 instances, where DP-N2M2 solved all instances within seconds per instance. In contrast, other methods suffered

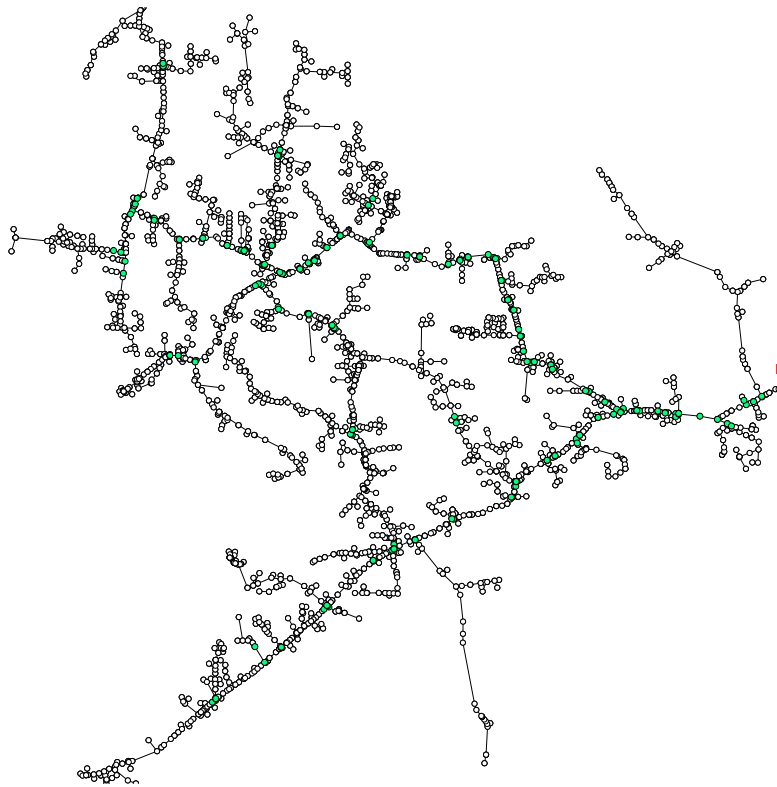


Fig. 5. Switch allocation (green) on network n3637 ($m = 100$).

from combinatorial explosion, limiting their applicability to smaller networks or instances with fewer switches.

The polynomial-time complexity of DP-N2M2 makes it uniquely scalable and practical for large-scale applications. By enabling the optimal allocation of switches even in extensive networks with thousands of nodes, our approach provides a feasible and efficient tool for power utilities to enhance distribution network reliability and reduce outage costs.

The dynamic programming approach presented here may be extended to accommodate more general constraints and objectives. For instance, hard restrictions on switch locations, arising from cost, zoning, or operational constraints, can be incorporated by fixing the corresponding decisions variables in the recurrence. Furthermore, the method extends naturally to weighted-sum objectives (e.g., combining ENS, SAIDI, SAIFI or costs), since the recurrence structure supports additive terms. This property enables the integration of multi-criteria formulations [26,27].

An important direction for future work is the extension of the proposed method to meshed or partially meshed distribution networks. While most distribution systems are operated radially, justifying the assumptions made in this work, modern utilities may adopt more flexible topologies to improve resilience or capacity. Extending the dynamic programming approach to such settings could potentially involve decomposition techniques or hybrid optimization frameworks to manage the increased structural complexity. More broadly, the computational complexity of SSAP under generalized formulations, such as meshed topologies or non-additive objectives, remains largely unexplored. Identifying tractable and intractable cases in this broader landscape represents an important research direction and complements the contribution of this paper.

CRedit authorship contribution statement

Fábio Luiz Usberti: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Project administration, Methodology, Investigation, Funding acquisition, Formal

analysis, Data curation, Conceptualization. **José Federico Vizcaino González:** Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Laura Silva de Assis:** Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Investigation. **Celso Cavellucci:** Writing – original draft, Validation, Methodology, Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Council for Scientific and Technological Development – CNPq, Brazil (grants 311907/2023-7 and 102199/2024-8).

Data availability

All data and source codes are available in a public repository.

References

- [1] G. Levitin, S. Mazal-Tov, D. Elmakis, Genetic algorithm for optimal sectionalizing in radial distribution systems with alternative supply, *Electr. Power Syst. Res.* 35 (3) (1995) 149–155, [http://dx.doi.org/10.1016/0378-7796\(95\)01002-5](http://dx.doi.org/10.1016/0378-7796(95)01002-5).
- [2] M.M. Aman, G.B. Jasmon, H. Mokhlis, A.H.A. Bakar, Optimum tie switches allocation and DG placement based on maximisation of system loadability using discrete artificial bee colony algorithm, *IET Gener. Transm. Distrib.* 10 (10) (2016) 2277–2284, <http://dx.doi.org/10.1049/iet-gtd.2015.0303>.
- [3] M. Izadi, A. Safdarian, A MIP model for risk constrained switch placement in distribution networks, *IEEE Trans. Smart Grid* 10 (4) (2019) 4543–4553, <http://dx.doi.org/10.1109/TSG.2018.2863379>.

- [4] K. Pereira, B.R. Pereira, J. Contreras, J.R.S. Mantovani, A multiobjective optimization technique to develop protection systems of distribution networks with distributed generation, *IEEE Trans. Power Syst.* 33 (6) (2018) 7064–7075, <http://dx.doi.org/10.1109/TPWRS.2018.2842648>.
- [5] F.L. Usberti, J.F.V. González, C. Cavellucci, A quadratic formulation for the optimal allocation of fault indicators, *Electr. Power Syst. Res.* 231 (2024) 110352, <http://dx.doi.org/10.1016/j.epsr.2024.110352>, URL: <https://www.sciencedirect.com/science/article/pii/S0378779624002402>.
- [6] A.J. Benavides, M. Ritt, L.S. Buriol, P.M. França, An iterated sample construction with path relinking method: Application to switch allocation in electrical distribution networks, *Comput. Oper. Res.* 40 (1) (2013) 24–32, <http://dx.doi.org/10.1016/j.cor.2012.05.006>.
- [7] H. Karimi, T. Niknam, M. Dehghani, M. Ghiasi, M. Ghasemigarpachi, S. Padmanaban, S. Tabatabaee, H. Aliev, Automated distribution networks reliability optimization in the presence of DG units considering probability customer interruption: A practical case study, *IEEE Access* 9 (2021) 98490–98505, <http://dx.doi.org/10.1109/ACCESS.2021.3096128>.
- [8] A. Norouzi, H. Shayeghi, J. Olamaei, Multi-objective allocation of switching devices in distribution networks using the modified barnacles mating optimization algorithm, *Energy Rep.* 8 (2022) 12618–12627, <http://dx.doi.org/10.1016/j.egy.2022.09.028>.
- [9] Z. Galias, Deterministic branch and cut algorithm for multi-objective optimization of protective device allocation in radial distribution systems, *IEEE Trans. Ind. Inf.* (2025) <http://dx.doi.org/10.1109/TII.2024.3507080>.
- [10] J.-H. Teng, Y.-H. Liu, A novel ACS-based optimum switch relocation method, *IEEE Trans. Power Syst.* 18 (1) (2003) 113–120, <http://dx.doi.org/10.1109/TPWRS.2002.807038>.
- [11] R. Billinton, S. Jonnavithula, Optimal switching device placement in radial distribution systems, *IEEE Trans. Power Deliv.* 11 (3) (1996) 1646–1651, <http://dx.doi.org/10.1109/61.517529>.
- [12] R. Pinho, F. Saraiva, A comparison of crossover operators in genetic algorithms for switch allocation problem in power distribution systems, in: 2020 IEEE Congress on Evolutionary Computation, CEC, 2020, pp. 1–8, <http://dx.doi.org/10.1109/CEC48606.2020.9185795>.
- [13] G.P. Epifanio, J.F.V. González, F.L. Usberti, L. Tarrataca, L.S. Assis, Switch allocation problem in power distribution systems with distributed generation, *Oper. Res. Forum* 4 (3) (2023) 246–253, <http://dx.doi.org/10.1007/s43069-023-00236-1>.
- [14] A.R. Yari, M.R. Shakarami, F. Namdari, H.M. CheshmehBeigi, A new practical approach to optimal switch placement in the presence of distributed generation, *Iran. J. Sci. Technol. Trans. Electr. Eng.* 44 (2) (2020) 989–1002, <http://dx.doi.org/10.1007/s40998-019-00284-6>.
- [15] A. Abiri-Jahromi, M. Fotuhi-Firuzabad, M. Parvania, M. Mosleh, Optimized sectionalizing switch placement strategy in distribution systems, *IEEE Trans. Power Deliv.* 27 (1) (2012) 362–370, <http://dx.doi.org/10.1109/TPWRD.2011.2171060>.
- [16] Ž. Popović, B. Brbaklić, S. Knežević, A mixed integer linear programming based approach for optimal placement of different types of automation devices in distribution networks, *Electr. Power Syst. Res.* 148 (2017) 136–146, <http://dx.doi.org/10.1016/j.epsr.2017.03.028>.
- [17] A. Heidari, Z.Y. Dong, D. Zhang, P. Siano, J. Aghaei, Mixed-integer nonlinear programming formulation for distribution networks reliability optimization, *IEEE Trans. Ind. Inform.* 14 (5) (2018) 1952–1961, <http://dx.doi.org/10.1109/TII.2017.2773572>.
- [18] M. Pau, F. Ponci, A. Monti, Optimal placement of sectionalizing switches for radial distribution grids, in: SyNERGY MED 2019 - 1st International Conference on Energy Transition in the Mediterranean Area, 2019, <http://dx.doi.org/10.1109/SyNERGY-MED.2019.8764121>.
- [19] Z. Galias, Tree-structure based deterministic algorithms for optimal switch placement in radial distribution networks, *IEEE Trans. Power Syst.* 34 (6) (2019) 4269–4278, <http://dx.doi.org/10.1109/TPWRS.2019.2909836>.
- [20] F.L. Usberti, C. Cavellucci, C. Lyra, Interruption flows for reliability evaluation of power distribution networks, *Oper. Res.* 2023 (23) (2023) 4, <http://dx.doi.org/10.1007/s12351-023-00758-w>.
- [21] R. Diestel, *Graph Theory*, Springer (print edition); Reinhard Diestel (eBooks), 2024.
- [22] R. Billinton, R.N. Allan, *Reliability Evaluation of Power Systems*, second ed., Plenum Press, New York, NY, 1996.
- [23] D.C. Mnyanghwalo, S.J. Kawambwa, Sensor placement algorithm for faults detection in electrical secondary distribution network using dynamic programming method: focusing on dynamic change and expansion of the network configurations, *J. Electr. Syst. Inf. Technol.* 11 (1) (2024) 13.
- [24] J.F. Vizcaino González, C. Lyra, F.L. Usberti, A pseudo-polynomial algorithm for optimal capacitor placement on electric power distribution networks, *European J. Oper. Res.* 222 (1) (2012) 149–156.
- [25] R. Pérez-Guerrero, G.T. Heydt, N.J. Jack, B.K. Keel, A.R. Castelhana, Optimal restoration of distribution systems using dynamic programming, *IEEE Trans. Power Deliv.* 23 (3) (2008) 1589–1596.
- [26] M. Khani, R. Ghazi, B. Nazari, Decision support system for optimal location of HIFDs in real distribution network using an integrated EPSO-fuzzy AHP model, *IET Gener. Transm. Distrib.* 14 (9) (2020) 1616–1626.
- [27] M. Khani, R. Ghazi, B. Nazari, An unsupervised learning based MCDM approach for optimal placement of fault indicators in distribution networks, *Eng. Appl. Artif. Intell.* 125 (2023) 106751.